

# Phase III Report: Academic Library Management System

Safal Shrestha  
shre4420@vandals.uidaho.edu  
University of Idaho  
Moscow, Idaho, USA

S Q Abdullah Al Mamun  
mamu2549@vandals.uidaho.edu  
University of Idaho  
Moscow, Idaho, USA

Swikriti Shrestha  
shre9317@vandals.uidaho.edu  
University of Idaho  
Moscow, Idaho, USA

## Abstract

The Academic Library Management System is a graph-oriented database prototype for storing and exploring academic library records. The system models authors, institutions, publications, and venues as graph nodes. It also models authorship, affiliation, publication venue, citation, and co-author relationships as graph edges. The Phase III implementation provides a working database schema, seeded academic dataset, Flask backend API, browser interface, and Cytoscape.js graph visualizations. The prototype supports the three required graph queries: one-hop co-author network visualization, h-index filtering for authors with h-index greater than or equal to five, and Q1 journal influence network discovery. The current version also includes a dashboard, collection browsing, search tools, and a demo login/signup screen for presenting the application as a functional prototype.

## CCS Concepts

• Information systems → Database design and models; Graph-based database models; • Human-centered computing → Graph drawings.

## Keywords

graph database, academic library, SQLite, Flask, Cytoscape.js, h-index, co-author network, Q1 journal

### ACM Reference Format:

Safal Shrestha, S Q Abdullah Al Mamun, and Swikriti Shrestha. 2026. Phase III Report: Academic Library Management System. In *Proceedings of Database Systems Phase III Project (CS 360 Phase III)*. ACM, New York, NY, USA, 5 pages.

## 1 Introduction

Academic library systems usually store bibliographic records in separate relational tables such as authors, publications, journals, and institutions. This structure is useful for record management, but it can make relationship-centered questions harder to answer. For example, a user may want to identify collaboration networks, find authors connected to influential journals, or evaluate an author's citation impact across publications.

This project interprets the academic library as a graph. Each real-world entity becomes a node, and each relationship becomes an edge. This makes the database better aligned with questions about academic influence, collaboration, and publication context. The system is implemented with SQLite relational tables, but the schema is graph-oriented. All entities are registered in a shared Nodes table, all relationships are stored in a shared Edges table, and graph membership is tracked through mapping tables.

The goal for Phase III is to demonstrate a functional prototype. The current application can load seeded data, expose graph query results through backend API routes, and visualize graph query output in the browser. The prototype is not yet a production library platform, but it demonstrates the database design, the required queries, and the user interface flow needed for the final phase.

## 2 Requirement Interpretation

The project specification requires a graph database system with node storage, edge storage, multi-graph support, and three query demonstrations.

For node storage, the system stores all node identities in one Nodes table and assigns each node a unique NodeID. The implementation supports four node types: Author, Institution, Publication, and Venue. Type-specific attributes are normalized into subtype tables. For example, author names, research areas, and emails are stored in Authors, while venue quartiles and impact scores are stored in Venues.

For edge storage, the system stores all graph relationships in one Edges table. This prototype supports AUTHORED, AFFILIATED\_WITH, PUBLISHED\_IN, CITES, and CO\_AUTHOR edges. These edges connect nodes of different or same types. For example, an author can be connected to a publication through an AUTHORED edge, a publication can be connected to a venue through a PUBLISHED\_IN edge, and two authors can be connected through a CO\_AUTHOR edge.

For multi-graph support, the implementation uses three graph-related tables: Graphs, NodeGraphs, and EdgeGraphs. This allows the same author, publication, venue, or edge to participate in different graph views, such as a collaboration graph, citation graph, or venue influence graph.

## 3 Technology Stack

The prototype uses a lightweight stack suitable for a class project and live demonstration.

The database is SQLite. SQLite was selected because it is easy to set up locally, works well with a seeded dataset, and supports the relational constraints needed to simulate graph storage.

The backend is implemented with Python and Flask. Flask serves both static frontend files and JSON API endpoints for graph queries. The backend is responsible for database initialization, query execution, and sending structured responses to the frontend.

The frontend is built with plain HTML, CSS, and JavaScript. Cytoscape.js is used for graph rendering. It displays co-author and Q1 influence networks directly in the browser.

Testing is done with Node's built-in test runner. The tests validate h-index filtering, co-author network results, collection queries, and Q1 influence network behavior.

## 4 Expanded System Architecture

The architecture has four main layers: the data layer, backend service layer, frontend application layer, and visualization layer.

The data layer is defined in `database/schema.sql`. It includes shared graph tables, subtype tables, graph membership tables, and indexes. The seeded dataset includes authors, institutions, publications, venues, citations, and co-author relationships.

The backend service layer is implemented in Flask. The backend initializes the database if needed, executes graph queries, and returns JSON data to the frontend. This layer acts as the connection between the browser interface and the SQLite database.

The frontend application layer presents the login/signup screen, dashboard, researchers page, collections page, analytics page, and search interface. The frontend calls backend API routes using `fetch()` and updates the page based on the returned data.

The visualization layer uses Cytoscape.js. The frontend receives graph-ready JSON from the backend, converts records into Cytoscape elements, and renders the selected graph view.

The user flow begins at the login/signup screen. After login, the dashboard loads summary statistics from `/api/summary`. The Researchers tab calls `/api/authors` and `/api/query/coauthors`. The Collections tab calls `/api/collections`. The Analytics tab calls `/api/query/h-index` and `/api/query/q1-influence`.

## 5 Database Design

The central design decision is to separate graph identity from entity-specific attributes. The Nodes table stores every graph node. Its main fields are `NodeID`, `NodeType`, `DisplayLabel`, and `CreatedAt`. The system currently includes four node types: authors, institutions, publications, and venues.

Subtype attributes are stored in separate tables. The Authors table stores author names, research areas, and emails. The Institutions table stores institution names and countries. The Publications table stores publication titles, years, and DOI values. The Venues table stores venue names, venue kind, quartile, and impact score.

Each subtype table uses `NodeID` as both its primary key and a foreign key to Nodes. This keeps the node identity consistent while preserving normalized storage for attributes.

The Edges table stores all graph relationships. Its main fields are `EdgeID`, `SourceNodeID`, `TargetNodeID`, `EdgeType`, `EdgeYear`, `Weight`, and `MetadataJson`. The supported edge types are `AUTHORED`, `AFFILIATED_WITH`, `PUBLISHED_IN`, `CITES`, and `CO_AUTHOR`.

The `Weight` field is especially useful for `CO_AUTHOR` edges because it stores the number of shared publications. The `EdgeYear` field stores temporal information such as publication year or latest collaboration year. `MetadataJson` allows optional extensible metadata without changing the schema.

Multi-graph support is implemented through `Graphs`, `NodeGraphs`, and `EdgeGraphs`. The seeded system includes three graph views: collaboration graph, citation graph, and venue influence graph.

## 6 Normalization Discussion

The schema is normalized to reduce duplication. The Nodes table stores common identity fields, while subtype tables store fields only relevant to a specific entity type. For example, `Quartile` is only meaningful for venues, so it is stored in Venues, not in Nodes.

Edges are also normalized into one table instead of separate relationship tables. This supports flexible graph traversal because all relationships can be queried from one place. The tradeoff is that different edge types may use fields differently. For example, `Weight` means shared publication count for `CO_AUTHOR`, but it is usually one for `AUTHORED` and `CITES`. The `EdgeType` field clarifies how to interpret each edge.

This design also makes the project easier to expand. New node types and new edge types can be added without redesigning the whole schema. For a classroom prototype, this gives the system enough flexibility while still keeping the database structure understandable.

## 7 Functional Prototype Description

The current prototype includes several user-facing screens.

The login/signup screen appears before the application. For the current prototype, user accounts and sessions are stored in browser `localStorage`. This is useful for the presentation flow but is not production-secure authentication.

The dashboard shows total counts for authors, publications, venues, and relationships. The current seeded totals are 8 authors, 15 publications, 6 venues, and 123 relationships.

The Researchers page supports the required co-author query. The user selects an author, and the page shows the selected author's profile, collaboration metrics, graph visualization, co-author table, and selected author publication list.

The Collections page shows publication records and venue cards. This page demonstrates how publications, authors, and venues are connected through graph relationships.

The Analytics page includes the h-index filter and Q1 journal influence network. This page demonstrates the second and third required queries.

The system also includes search tools for authors, publications, institutions, and venues. Institution search can display authors affiliated with a selected institution.

## 8 Required Query Implementation

### 8.1 Query 1: Co-Author Network

The co-author network query is implemented by retrieving `CO_AUTHOR` edges for a selected author. Given an author ID, the backend first confirms the selected author exists. It then retrieves `CO_AUTHOR` edges where the selected author is either the source or target. The connected author IDs become the one-hop network.

The query returns the selected author profile, graph nodes, graph edges, direct co-author count, publication count, total shared works, strongest collaboration, collaborator table data, and selected author's publication table data.

The frontend visualizes this response with Cytoscape.js. The selected author is highlighted separately from co-authors. Edge labels show shared publication counts.

For example, Alice Carter has 5 direct co-authors, 7 publications, 8 shared works, and her strongest collaboration is Chloe Zhang with 3 shared publications.

## 8.2 Query 2: H-Index Filter

The h-index query identifies authors with h-index greater than or equal to five. The system computes citation count by counting incoming CITES edges for each publication. For each author, the backend gathers all authored publications and their citation counts. It then computes h-index using the standard method: an author has h-index  $h$  if the author has at least  $h$  publications with at least  $h$  citations each.

The current seeded dataset returns two authors that meet the threshold: Alice Carter and Chloe Zhang. Both authors have an h-index of 5. Their example venues include Journal of Graph Analytics, Network Science Letters, and Scholarly Data Mining Journal.

The Analytics page displays these authors, their h-index values, publication venues, and top publications.

## 8.3 Query 3: Q1 Journal Influence Network

The Q1 influence query identifies authors linked to authors publishing in Q1 journals. A Q1 journal is defined as any venue record where `Venues.Quartile = 'Q1'`. The query first finds authors who have authored publications published in Q1 venues. Then it checks `CO_AUTHOR` edges to identify authors connected to those Q1 publishers.

The returned graph includes authors who publish in Q1 venues, authors directly connected to Q1 publishers, and co-author edges among the network.

The current seeded dataset identifies eight qualifying authors in the Q1 influence network: Alice Carter, Ben Ortiz, Chloe Zhang, Daniel Kim, Emma Patel, Farah Nasser, Grace Liu, and Henry Brooks.

The Analytics page displays the qualifying author list and a Cytoscape graph visualization.

## 9 Implementation Details

The database initialization process is handled by `ensure_database()`. If the SQLite database does not exist or is empty, the system creates the schema and inserts seed data.

The backend exposes several API routes. The main routes are shown below:

- `/api/summary`
- `/api/authors`
- `/api/collections`
- `/api/query/coauthors`
- `/api/query/h-index`
- `/api/query/q1-influence`
- `/api/search/authors`
- `/api/search/publications`
- `/api/search/institutions`
- `/api/search/venues`

The frontend calls these endpoints using JavaScript `fetch()` requests. Graph endpoints return nodes and edges in a format that can be converted directly into Cytoscape elements. The graph renderer uses a force-directed layout and separate styling for selected authors, co-authors, Q1 publishers, and influenced authors.

The current login/signup system is implemented entirely in frontend JavaScript using `localStorage`. This was added to make the prototype feel like a complete application during the presentation.

In the final version, authentication should move to the backend with password hashing and server-managed sessions.

## 10 Preliminary Results

The prototype demonstrates that the graph-oriented schema supports the required academic queries. The dashboard confirms that the seeded graph contains 8 authors, 15 publications, 6 venues, and 123 relationships.

**Table 1: Seeded Dataset Summary**

| Metric        | Count |
|---------------|-------|
| Authors       | 8     |
| Publications  | 15    |
| Venues        | 6     |
| Relationships | 123   |

The co-author query successfully visualizes one-hop collaboration networks. The h-index query correctly identifies authors meeting the threshold of five. The Q1 influence query combines venue metadata with co-author edges, showing that the graph can answer questions involving multiple entity and edge types.

The automated tests currently cover h-index query output, co-author network output, collections query output, and Q1 influence network output. All tests pass in the current implementation.

## 11 Challenges

One challenge was designing a relational schema that behaves like a graph database while still satisfying the project requirement for Nodes and Edges tables. The solution was to keep all graph identity and relationships centralized, while normalizing type-specific attributes in separate tables.

Another challenge was computing graph analytics from relational tables. The h-index query requires joining authors, publications, venues, and citation edges. The Q1 influence query requires combining venue metadata with co-author relationships. These queries confirmed the value of indexing edge type, source node, and target node.

The frontend graph visualization also required careful handling. Cytoscape.js selectors match node data fields, so false flags had to be filtered or queried correctly to avoid coloring all co-author nodes as selected authors.

## 12 Limitations

The current dataset is seeded and small. This is appropriate for a classroom prototype, but larger datasets would be needed to evaluate performance and usefulness in a real academic library setting.

The authentication system is demo-only. It stores users in browser `localStorage` and should not be treated as secure. A production version would require backend authentication, password hashing, server-side sessions, and access control.

The current graph analytics are focused on the required queries. The system does not yet include advanced graph algorithms such as shortest path, centrality, clustering, or community detection.

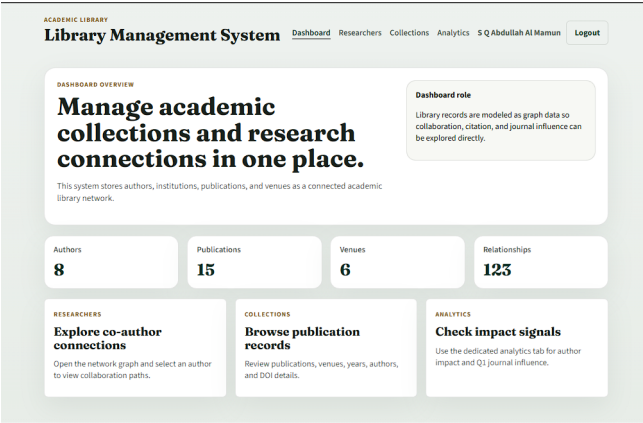


Figure 2: Dashboard page showing total authors, publications, venues, and relationships.

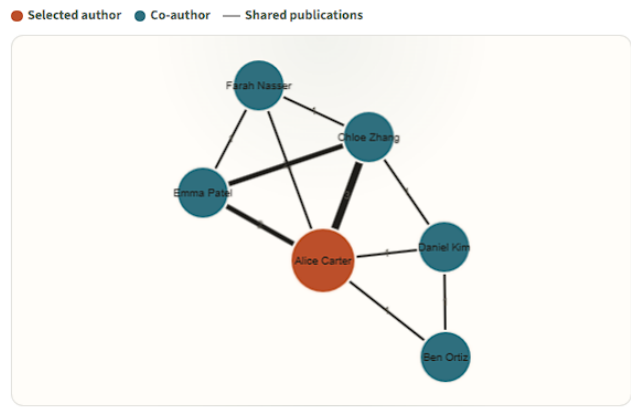


Figure 3: Researchers page showing the selected author and one-hop co-author network visualization.



Figure 4: Researchers page showing co-author list and selected author publication list.

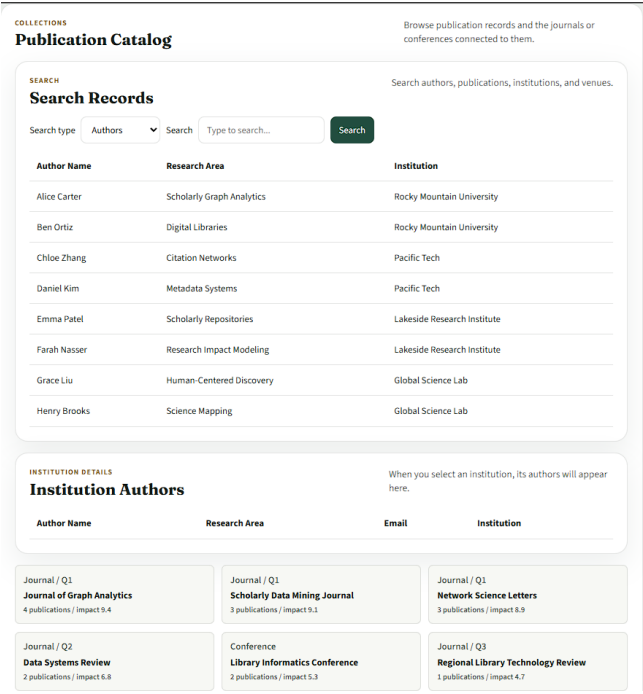


Figure 5: Collections page showing publication records and venue information.

The UI provides visualizations and tables, but it does not yet support exporting results to CSV or PDF.

### 13 Phase IV Plan

For the final phase, the main goals are to finalize the report and include screenshots and diagrams, polish the live demo flow, add stronger input validation and error messages, expand the seeded dataset if time allows, consider export functionality for query results, improve authentication or clearly document it as a demo feature, and add more tests around search and API error cases.

### 14 Conclusion

The Phase III prototype successfully implements the required graph database structure and query functionality. The system stores all node identities in a shared Nodes table, stores all relationships in a shared Edges table, and supports multi-graph membership through mapping tables. It demonstrates all three required graph queries with working frontend views and Cytoscape.js visualizations. The current application is ready for a Phase III live demo and provides a strong foundation for the final Phase IV prototype.

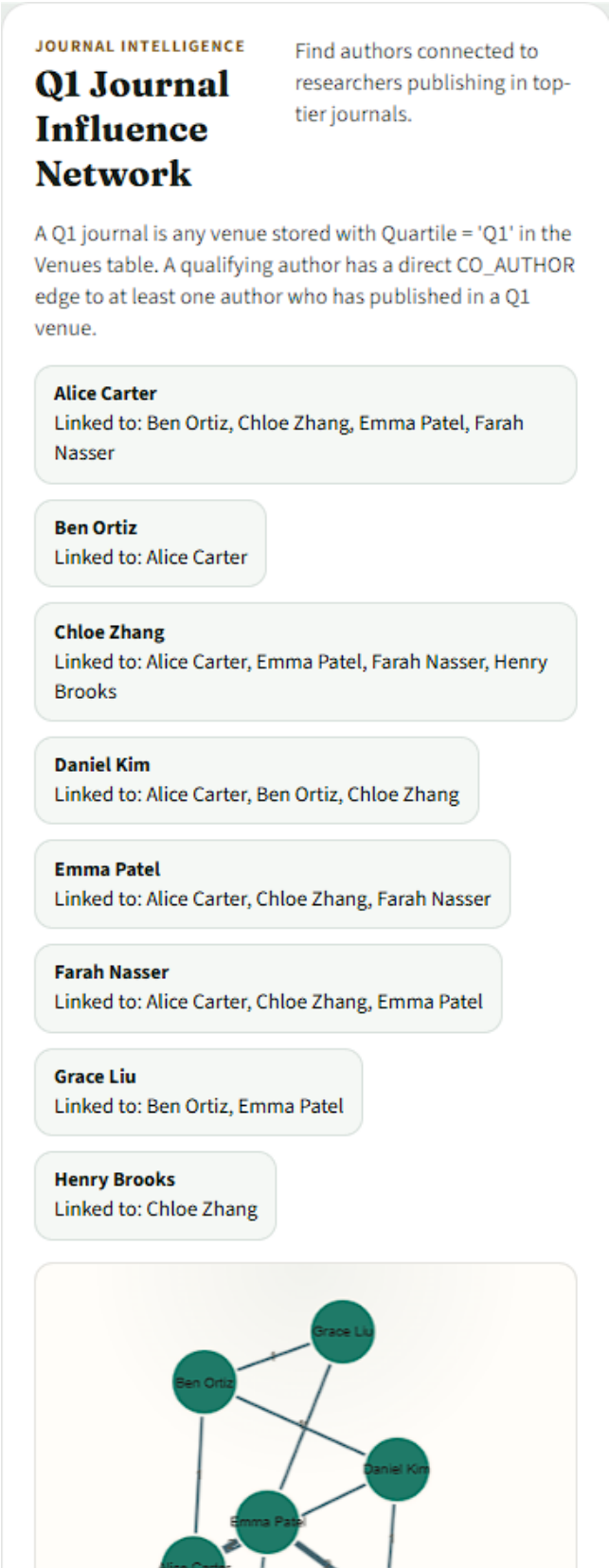


Figure 7: Analytics page showing the Q1 journal influence network.

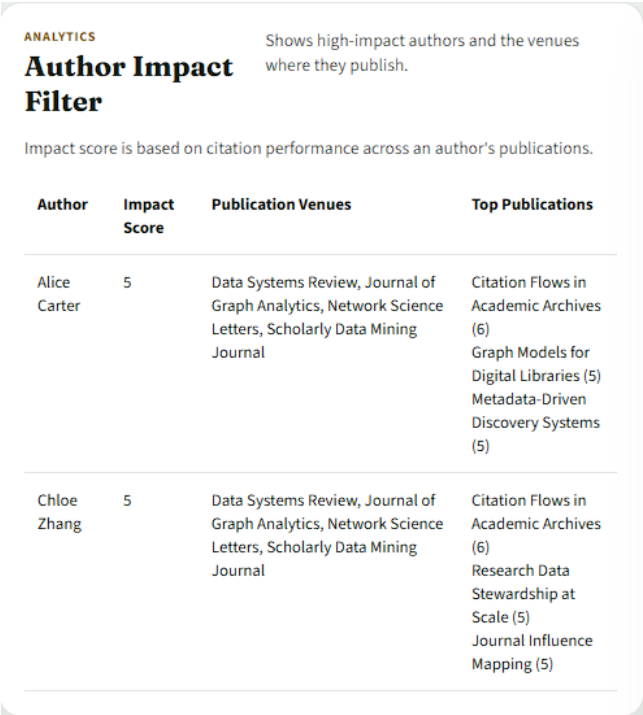


Figure 6: Analytics page showing authors returned by the h-index query.

A    Prototype Screenshots

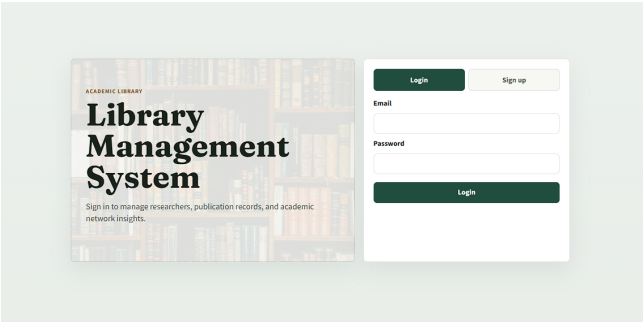


Figure 1: Login and signup screen for the Academic Library Management System prototype.